

# AXON FLOW

A Performant, Browser-Native Mind-Mapping Platform  
with Client-Side D3 Geometry and Optimistic UI

---

**Prepared By:**

Akshat Raj

(Reg. No: 23scse1010833)

**Collaborator:**

Pavan Soni

(Reg. No: 23scse1012576)

**Institution:**

Department of Computer Science & Engineering  
Galgotias University  
Greater Noida, India

June 2, 2026

# 1 Executive Summary

Brainstorming and knowledge mapping tools have transitioned from analog whiteboards to critical collaborative software. However, existing modern platforms (such as Miro, XMind, and GitMind) suffer from performance limitations, severe interface latency, and security concerns from pushing private brainstorming data into cloud AI clusters.

**Axon Flow** solves these problems by decoupling data representation from rendering layout. By combining a flat independent node model with local D3-Hierarchy calculations and an instantaneous Optimistic UI feedback loop, Axon Flow delivers a desktop-grade, zero-latency collaborative canvas running entirely in a standard browser. Additionally, it integrates a local, private Generative AI engine that runs offline, ensuring user data privacy.

## 1.1 What is Axon Flow? (Non-Technical Key Points)

To introduce the project in simple, clear terms, here are the core concepts:

- **Infinite Digital Notebook:** Axon Flow is an interactive web-based mind map where users can brainstorm and connect ideas visually on an infinite canvas.
- **Instant UI Updates:** Unlike other tools that freeze or show loading spinners on every change, Axon Flow updates the screen instantly (0ms delay) and saves changes silently in the background.
- **Text and Mind-Map Dual View:** It bridges the gap between visual diagrams and linear reading by allowing users to toggle their maps into structured text outlines (like a reading tutorial).
- **Local, Secure AI:** Users can ask the built-in AI assistant to auto-expand branches or outline topics offline, keeping their brainstorming private.
- **Unified Study Hub:** Each node on the map acts as a repository where users can store Markdown notes, status indicators, hyperlinks, and upload reference files.

### Value Proposition

Axon Flow offers an **instantaneous (0ms perceived latency)** visual mind-mapping canvas that handles over 500 nodes at 60 FPS, features a linearized reading mode to prevent canvas fatigue, and maintains strict user privacy with local LLM capabilities.

## 2 Market Pain Points & The Problem

Modern digital mind-mapping tools exhibit three core bottlenecks that hinder productivity:

- 1. Nested-Document Database Overheads:** Standard tools store entire mind-map trees as a single nested document (JSON) in the database. When a user updates, moves, or renames a single node, the entire hierarchy must be sent over the network and rewritten in the database. This leads to heavy database lock times, write-amplification, and synchronization conflicts in multi-user settings.
- 2. Canvas Fatigue and Layout Disorientation:** As a mind-map grows beyond 50 nodes, users suffer from "canvas fatigue." Continuously panning and zooming to read notes or edit leaf structures becomes disorienting. Traditional tools fail to offer a cohesive way to consume deep visual structures linearly.
- 3. Heavy DOM Rendering and UI Latency:** Tools that render canvas bubbles using hundreds of nested HTML `div` tags trigger browser reflows and layout recalculations on every drag event. This degrades frame rates (<15 FPS) and introduces noticeable lag. Alternatively, server-side layouts introduce a delay (200ms–1.5s) on every user action while waiting for the database confirmation.

## 3 Technical Solution & Innovation

Axon Flow solves these fundamental constraints with a modern, high-performance architecture:

### 3.1 Flat-Tree Data Model

Rather than nested JSON documents, every node in Axon Flow is an independent document in MongoDB. The document tracks only three primary keys: 'id', 'mapId', and 'parentId'.

Update Complexity:  $O(1)$  database update query

Updating a node's name, content, or collapsing a branch modifies only that single record via a lightweight PATCH request, avoiding database write-amplification.

### 3.2 Client-Side D3 Geometry calculations

The server acts as a lightweight data dispenser, sending a flat list of nodes to the client. The frontend then reconstructs the visual tree in memory using D3-Hierarchy's layout stratify algorithms in under **1 millisecond** for 500 nodes. Nodes are positioned dynamically based on label width calculations.

### 3.3 Zero-Latency Optimistic UI

Axon Flow updates the UI state instantly based on expected success. When a node is added, renamed, or reparented via drag-and-drop:

- a) The local React state is updated immediately (0ms delay).
- b) D3 math computes coordinates in the background.
- c) A silent network request is dispatched.
- d) If the request fails, the local state is rolled back and the user is notified.

## 4 Product Features & Functionality

- **Document-Wise Reading Mode:** Toggles the visual layout into a structured text document. The D3 visual tree is linearized into hierarchical headings (H1, H2, H3) based on depth, displaying markdown notes and code attachments inline. This eliminates panning and zoom fatigue.
- **Radial Status Indicators:** Visual toggle pins on each node allow students and

developers to track learning/completion states (*Reading, Completed, Revising, Important*) directly from the canvas or the outline view.

- **Offline Local AI Assistant:** A FastAPI service utilizing Ollama and LangChain streams AI topics (SSE protocol) to automatically expand node trees based on prompt topics, operating strictly on the user's local machine for absolute data privacy.

## 5 The User Journey & Non-Technical Modules

To understand how Axon Flow works in practice, let us look at the platform through a universal user workflow of building, enriching, and utilizing a mind map:

### 5.1 The Universal User Workflow

#### a) Instant Brainstorming (The Map Canvas):

- The user opens Axon Flow and is presented with a clean, **infinite digital canvas**.
- With a simple **double-click**, a new node bubble is created instantly on the workspace with **zero network delay (0ms)**.
- The user types a topic label, and the bubbles automatically slide apart mathematically to maintain neat coordinates.

#### b) Creating Nodes, Adding Files & Resource Links:

- The user creates child branches and **uploads reference PDFs, slides, or code files** directly to any node.
- The user embeds **external hyperlinks** (such as project repositories or online videos) directly inside a node's detail panel.
- Rich **Markdown notes** are typed in the inline text sidebar, consolidating all media resources in one visual hub.

#### c) AI-Powered Node Expansion:

- To automatically expand a concept, the user right-clicks a node and triggers the **local AI assistant**.
- An **offline Llama 3 model** running locally on the user's laptop brainstorms 4–6 relevant subtopics as children.
- These branches sprout instantly on the canvas, keeping the user's proprietary notes **100% private and secure**.

#### d) Linearized Outline Mode (Eliminating Canvas Fatigue):

- When the visual canvas becomes too crowded to easily read, the user toggles the **Document-Wise Reading View**.
- The hierarchical D3 visual structure is instantly flattened into **sequentially organized text headings** (H1, H2, H3).
- Notes, file attachments, and code snippets render inline, reading naturally like a **digital notebook chapter**.

#### e) Interactive Progress Tracking (Radial Status Badges):

- Next to each header or node bubble, the user selects a **radial button** to update their study or task state.
- The user toggles status badges across states such as ***Completed*** (✓), ***Reading*** (R), or ***Revising*** (!).
- Status changes sync instantly in the background, updating both the visual canvas and document outlines simultaneously.

## 5.2 Non-Technical Core Modules

Under the hood, Axon Flow is split into four simple user-facing modules:

- **The Infinite Creative Canvas:** A drag-and-drop workspace where ideas can be **visually structured, linked, and organized** without bounds.
- **The Smart Reader (Document Mode):** A linearized text workspace that turns visual diagrams into **organized outline guides** with inline attachments.
- **The Private AI Brainstormer:** A local copilot that helps flesh out subtopics and **auto-generate mind maps offline** without cloud risks.
- **The Study & Resource Vault:** A hub attached to every node where **files, hyperlinks, and status badges** are securely stored.

## 6 Competitive Matrix

The following table highlights how Axon Flow compares to the leading products in the market:

| Dimension          | Traditional Tools<br>(Miro/XMind)                          | Axon Flow                                                          |
|--------------------|------------------------------------------------------------|--------------------------------------------------------------------|
| Data Privacy       | High risk; data uploaded to cloud LLMs (OpenAI/Anthropic). | <b>100% private</b> ; runs local model offline via Ollama.         |
| Code Integrations  | Plain text only; no developer attachment capability.       | Bind <b>source code</b> , <b>attachments</b> , and links to nodes. |
| Reading Experience | Panning/zooming fatigue on deep visual maps.               | Dual view: Switch to linear <b>Document-Wise Outlines</b> .        |
| UI Performance     | Heavy canvas redraws causing lag during drags.             | Flat MongoDB schema + React Virtual DOM translation.               |
| Perceived Latency  | 200ms–1.5s (blocking network spinners).                    | <b>0ms (Optimistic UI background syncing)</b> .                    |

Table 1: Product Comparison Matrix

## 7 Architecture & Implementation Plan

Axon Flow is designed around the "**Smart Client, Dumb Server**" paradigm. The backend (Node.js/Express) serves only to store independent relational records, validate tokens via Clerk, and handle file attachments in S3.

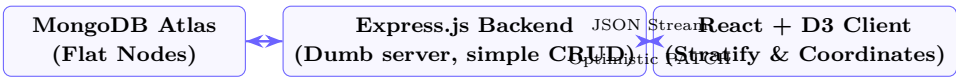


Figure 1: Data Flow Architecture

The project is structured for a three-phase rollout:

- ✓ **Phase 1 (Core Canvas):** Client-side layout parsing and flat schema updates.
- ✓ **Phase 2 (Optimistic UI & Files):** Implementation of drag-and-drop hit tests, offline rollbacks, and file uploads.
- ✓ **Phase 3 (Local AI & Stream):** FastAPI integration with local Ollama SSE stream nodes.



## 8 Project Key Points & Quick Summary

### 8.1 About My Project

- Performant browser-native mind mapper
- Decentralized flat-data structure model
- High-speed mathematical client layout
- Server-Sent Events (SSE) token generator
- Zero-latency optimistic canvas updates
- Offline local LLM expansion
- Linearized reading dual outlines

### 8.2 Key Features

- **0 Perceived Latency:** Predictive local node updates
- **Optimistic State Sync:** Background Mongo saves with rollback protection
- **Sub-millisecond D3 Stratification:** In-memory JSON tree rebuilding
- **Linearized Outline Mode:** Flat heading outlining (H1, H2, H3)
- **Offline Local Copilot:** Topic expansion using Ollama models
- **SSE Typewriter Stream:** Unidirectional chunk-by-chunk token push
- **Visual Status Badging:** Task indicators (✓, Reading, Revision, Important)
- **Media Node Vault:** PDF uploads, custom hyperlinks, notes attachments
- **Native Drag/Resize Listeners:** Lightweight mouse listeners
- **Microservices Auth Gateway:** Clerk-integrated backend checks

### 8.3 Core Modules

- **Infinite Creative Canvas:** SVG coordinate dragging and layout grid
- **Smart Outline Reader:** Monolithic map linearized outline guide
- **Offline AI Brainstormer:** Direct LLM inference node stream
- **Resource Repository:** File and link storage per node
- **Authentication Gateway:** Node.js request router and token check
- **Business Core Engine:** Spring Boot MVC backend

### 8.4 Technology Stack

- **Frontend Base:** React, Vite, ES6 JS
- **UI Canvas Layout:** React Flow
- **Layout Math Brain:** D3-Hierarchy (tree, stratify)
- **Gateway Route Manager:** Node.js, Express, JWT
- **Core Persistence Service:** Java, Spring Boot, Spring Data MongoDB
- **Database Storage System:** MongoDB Atlas
- **AI Execution Runtime:** Python, FastAPI, EventSource API

- **AI Orchestration Framework:** LangChain, Ollama Client
- **Local AI Models:** Llama 3 / DeepSeek R1
- **File Storage Space:** S3 / Local Disk

## 8.5 Why This Project? (Motivation)

- **Prevent AI Cloud Leaks:** Keep sensitive client mapping data offline
- **Solve Nested JSON Locks:** Flat collection maps avoid monolithic writes
- **Solve Canvas Repaint Lags:** SVG layout calculation renders 500+ nodes at 60 FPS
- **Prevent Pan/Zoom Canvas Fatigue:** Quick conversion to outlines
- **Deliver Seamless Latency-Free UX:** Ghost nodes remove load spinners
- **Master Polyglot Microservices:** Combine Spring, Node, Python architectures

## 9 Project Quick Reference & Learning Guide

To assist in understanding, defending, and discussing the technical architecture of Axon Flow, the key concepts, features, and engineering challenges are summarized below keyword-wise.

### 9.1 Core Project Concepts & Architecture (Keyword-Wise)

- **Polyglot Microservices Topology:** The architecture decouples tasks across different runtimes matching specific computational bounds:
  - *Frontend (React + Vite):* Renders the interactive UI and computes layout geometry.
  - *Backend (Spring Boot + MongoDB):* Manages database transactions, permissions, and node relational persistence.
  - *AI Engine (FastAPI + Python):* Orchestrates local LLM inference, LangChain tools, and SSE streaming.
- **I/O Gateway vs. CPU Processing:** Node.js handles I/O-bound API routing (Gateway) via its single-threaded non-blocking event loop, whereas Spring Boot handles transactional integrity, and Python handles CPU/GPU-bound local LLM inference.
- **Flat-Tree Schema Representation:** Relational node records are stored flat in MongoDB via {id, parentId, mapId}, preventing write amplification, database locks, and sync conflicts common in monolithic JSON trees.
- **Data Driven Document (D3) Geometry:** The client calculates mathematical coordinate layouts in-memory using D3-Hierarchy (`d3.stratify()` and `d3.tree()`) in under 1ms, decoupling layout computation from DOM rendering.

### 9.2 Key Product Features (Keyword-Wise)

- **Optimistic UI Synchronization:** Updates local React state instantly to render node operations, running coordinate layouts in the background and syncing via background network requests (0ms perceived latency).
- **Linearized Document-Wise Outlines:** Translates the two-dimensional D3 canvas nodes into hierarchical text outlines (H1, H2, H3) inline with rich-text Markdown notes, eliminating canvas pan/zoom fatigue.
- **Offline Local AI Assistant:** Auto-generates and expands concept trees using local LLMs (via Ollama) running offline on the user's system, guaranteeing absolute privacy.

- **Server-Sent Events (SSE):** Streams AI-generated tokens to the client character-by-character, preventing blocking page loads and displaying live, typewriter-like thinking text.
- **Radial Status Badge Controls:** Interactive visual indicators (*Reading, Completed, Revising, Important*) that sync in real time between the visual canvas and linear document outlines.

### 9.3 Technical Challenges & Solutions (Keyword-Wise)

- **Challenge: Nested JSON Serialization Failures (Jackson Serialization)**
  - *Problem:* Dynamic node metadata (like lists of links and files) failed to deserialize properly at the Spring Boot backend, collapsing to generic objects or causing null values.
  - *Solution:* Changed the Java persistence entity field structure to `List<Map<String, Object>>` with explicit null-guard setters to ensure type safety.
- **Challenge: Broken JSON from Local LLMs (JSON Extraction)**
  - *Problem:* Smaller offline models (like Llama 3) frequently emit incomplete JSON blocks or surround response streams in markdown wrappers, breaking frontend parsers.
  - *Solution:* Developed a multi-stage parser in the Python AI Engine using regular expressions, curly brace counting, and automated repairs to sanitize and parse malformed streams.
- **Challenge: Heavy DOM Reflows during Canvas Dragging (UI Lag)**
  - *Problem:* Redrawing coordinate connections and updating node elements using standard React state updates triggers browser paint/reflow loops, dropping frame rates.
  - *Solution:* Integrated native D3 geometry math to run calculations in-memory, updating coordinates directly via SVG properties while leaving the React Virtual DOM unaffected.
- **Challenge: Drag-and-Drop Library Bloat**
  - *Problem:* Importing third-party canvas libraries (e.g. react-dnd) increases bundle size and introduces unnecessary configuration overhead.
  - *Solution:* Implemented a native mouse event listener (`mousedown`, `mousemove`, `mouseup`) bound to the window to calculate delta coordinates.

## 10 Interview Questions & Answers

### Question 1: D3 vs React Flow

For making a deep nested hierarchy, is this step a must (like using D3 inside the pipeline)? Or is there any shortcut in React Flow?

### Answer

**Yes, using an external layout engine like D3 (or Dagre/ELK) is completely necessary.** React Flow does not support automatic hierarchical layout calculations out-of-the-box.

- **React Flow is a Dumb Canvas:** React Flow only physicalizes nodes on a coordinate grid. If coordinates are not provided, all nodes pile up at (0,0). It lacks tree layout calculation logic.
- **D3-Hierarchy is the Brains:** We use D3 to construct the nested hierarchy in-memory (`d3.stratify()`) and run advanced node spacing algorithms (`d3.tree()`) to calculate perfect non-overlapping coordinates.

### Question 2: Polyglot Microservices Selection

Why did you choose a Polyglot Microservices architecture (Node.js API Gateway, Java/Spring Boot Core Backend, and Python FastAPI AI Engine) instead of a monolithic single-language setup?

### Answer

This structure aligns each service with its exact computational profile, ensuring optimal performance:

- **Node.js API Gateway (I/O Bound):** Handles concurrent connection routing and token verification. Its non-blocking, single-threaded event loop handles high request volumes with low memory footprint.
- **Java/Spring Boot Core Backend (Logic Bound):** Handles hierarchical operations and relational persistence. Java's strong typing, OOP design, and robust transaction management ensure structural integrity.
- **Python FastAPI AI Engine (CPU/GPU Bound):** Handles LLM prompt construction, agent workflows, and token parsing. Python is the industry standard for AI, and FastAPI supports asynchronous SSE streaming out-of-the-box.

### Question 3: Google Login (OAuth2) Flow

How does Google Login (OAuth2) authentication work in this polyglot microservice architecture, and how do you handle tokens?

#### Answer

We use Google as an identity validator, but manage API security via a custom system token:

- **Step 1 (Google Token):** React logs in via Google and receives a short-lived **Google ID Token**.
- **Step 2 (Verification):** React sends the token through the Node Gateway to Spring Boot. Spring Boot uses `GoogleIdTokenVerifier` to confirm its authenticity.
- **Step 3 (App JWT):** If valid, Spring Boot registers the user in MongoDB, discards the Google Token, and generates a signed custom **App JWT**.
- **Step 4 (Subsequent Requests):** React stores the App JWT and sends it in the Authorization header. The Node Gateway validates this App JWT locally for all microservice routing.

### Question 5: Client-Side Layout & Optimistic UI

How do you optimize canvas rendering performance and minimize perceived layout latency on the client side?

#### Answer

We eliminate lag through decoupled math rendering and predictive state updates:

- **Decoupled Calculations:** Layout math runs programmatically in D3 based on text-label width calculations, completely independent of the DOM paint pipeline.
- **Optimistic State Updates:** The UI updates and draws coordinates instantly upon node insertion, deletion, or moves. The network request is made in the background, rolling back state only on server errors.

### Question 6: Software Engineering Principles Alignment

How does the Axon Flow codebase align with modularity, Single Responsibility Principle (SRP), and clean code guidelines?

## Answer

The codebase is structured around strict separation of concerns to support scalability:

- **Modularity:** Adheres to Controller-Service-Repository patterns in Spring Boot and keeps UI widgets separated from state-management services in React.
- **Single Responsibility (SRP):** Layout geometry is isolated from React rendering; API operations are abstracted into custom hooks; and LLM parsing logic is handled strictly by Python's helper scripts.
- **Clean Code:** Clutter is reduced by moving inline CSS elements to external stylesheets and refactoring large rendering files into smaller, reusable UI blocks.

## 11 Conclusion

Axon Flow redefines web-based brain-mapping by utilizing client-side computing power. By eliminating blocking network spinners and nested storage patterns, it brings native-level desktop responsiveness directly to the browser, while respecting modern data privacy mandates through local generative models.